

A Reproducibility Study of CACHEUS for Adaptive Page and Cache Replacement

A Partial Reproduction Study of Learning Cache Replacement with CACHEUS

Shirin Manzari

M.Sc. Student in Computer Science, Software Engineering

Advanced Operating Systems

Course Instructor: Dr. Ravanmehr



Islamic Azad University, Central Tehran Branch

Spring 2026

Abstract

This paper documents a partial reproduction study of the FAST 2021 paper Learning Cache Replacement with CACHEUS. The original work proposed CACHEUS, a machine-learned cache replacement framework that uses online learning to select between two complementary eviction experts. The central claim of the original study is not that CACHEUS wins every individual workload, but that it is the most consistently strong choice across diverse storage workloads and cache sizes. The original evaluation covered 329 one-day workloads, five production I/O datasets, six cache configurations, and 17,766 simulation experiments.

The reproduction described here is intentionally narrower. It evaluates six cache replacement algorithms - LRU, LFU, ARC, LIRS, LeCaR, and CACHEUS - only on the CloudVPS dataset. The experiment uses 18 CloudVPS traces, five cache-size fractions, and 540 total algorithm runs. The workflow decodes raw CloudVPS block traces with blkparse, filters queue events, converts the result to the simulator .blk format, expands requests to page-level accesses, replays the traces through the cache simulator, and generates CSV summaries and PNG figures.

The main result is that CACHEUS remains highly competitive but is not the absolute winner on average in this CloudVPS-only reproduction. LIRS obtains an average hit rate of 28.08%, while CACHEUS obtains 28.06%, a difference of about 0.017 percentage points. This is practically a tie. CACHEUS still clearly improves over ARC, LeCaR, LRU, and LFU. These results partially support the original paper: CACHEUS is a strong general-purpose adaptive policy, but the CloudVPS subset also shows that LIRS can be extremely competitive for this particular workload family.

Table of Contents

1. [Introduction](#)
 2. [Background and Motivation](#)
 3. [Overview of the Original CACHEUS Paper](#)
 4. [Reproduction Study Design](#)
 5. [Project Pipeline Overview](#)
 6. [Technical Documentation of the Implementation](#)
 7. [Experimental Results](#)
 8. [Interpretation and Discussion](#)
 9. [Original Paper vs. Reproduction](#)
 10. [Threats to Validity and Limitations](#)
 11. [Recommendations for Future Work](#)
 12. [Conclusion](#)
- [References](#)

1. Introduction

Caching is one of the most important performance techniques in computer systems. A cache stores recently or frequently used data closer to the processor, application, storage client, or network edge so future requests can be served faster. In storage systems, a good cache can reduce latency, reduce backend I/O pressure, and improve the apparent throughput of applications. However, a cache has limited capacity. When the cache is full and a new object must be admitted, the system must choose which resident object to evict. This decision is called cache replacement.

The difficulty of cache replacement is that workloads are not stable. Some workloads reward recency because recently accessed items are likely to be reused soon. Other workloads reward frequency because a few popular items dominate access behavior. Some workloads contain scans: long sequences of one-time accesses that should not destroy the useful cache state. Others contain churn: repeated access over a working set larger than the cache, causing simple policies to replace data that will soon be needed again. A single static policy can perform very well under one pattern and poorly under another.

Traditional cache replacement policies such as LRU and LFU are simple and still useful, but they encode fixed assumptions. LRU assumes recent items are valuable. LFU assumes frequent items are valuable. More advanced policies such as ARC and LIRS were designed to address important weaknesses of these simple policies, but even those algorithms can behave inconsistently when workload characteristics or cache sizes change. The CACHEUS paper responds to this problem by treating cache replacement as an adaptive online decision problem.

This report has two goals. First, it explains the main idea of the original CACHEUS paper in a way that is useful for a presentation or project defense. Second, it documents the reproduction workflow and results obtained from a CloudVPS-only study. The purpose is not to claim a full reproduction of every result in the original paper. Instead, this work should be read as a focused reproduction and engineering study that checks whether the original claims remain directionally visible when the scope is reduced to one dataset family.

2. Background and Motivation

2.1 Cache Replacement and Hit Rate

A cache hit occurs when the requested data is already present in the cache. A cache miss occurs when the requested data is absent and must be fetched from a slower layer. The primary metric in this study is hit rate. Hit rate is calculated as hits divided by total requests, often expressed as a percentage: $\text{hit rate} = \text{hits} / (\text{hits} + \text{misses}) * 100$. Higher hit rate usually means the cache is more effective, although runtime overhead and implementation complexity also matter.

In storage trace simulation, the replacement algorithm is evaluated by replaying a recorded sequence of requests. For each request, the simulator asks the algorithm whether the page is present. On a hit, the algorithm updates its internal metadata. On a miss, the algorithm may insert the new page and evict an old page if the cache is full. The hit count and miss count collected during replay provide the final hit rate for that trace, cache size, and algorithm combination.

2.2 Why One Policy Is Not Enough

The original CACHEUS paper argues that modern storage workloads are mixtures of several primitive behaviors. The important observation is that these primitive behaviors can appear across different workloads, inside the same workload over time, and at different cache sizes. This matters because cache size changes the meaning of locality. A sequence that looks easy for LRU when the cache is large may become churn-like when the cache is too small to retain the working set.

The four primitive workload types used by CACHEUS are LRU-friendly, LFU-friendly, scan, and churn. LRU-friendly behavior is best handled by recency. LFU-friendly behavior is best handled by frequency. Scan behavior should not pollute the cache with one-time data. Churn behavior requires resistance to repeated eviction from a

working set that is larger than the cache. These categories are not mutually exclusive, but they are useful because each one points to a different kind of eviction decision.

Primitive type	Main behavior	Why it matters
LRU-friendly	Recently used pages are likely to be reused.	Recency-based eviction can work well.
LFU-friendly	Frequently used pages dominate future requests.	Frequency-based protection is useful.
Scan	Many pages are accessed once.	The cache should avoid pollution by one-time pages.
Churn	A working set larger than cache is repeatedly accessed.	Naive recency can repeatedly evict useful data.

3. Overview of the Original CACHEUS Paper

3.1 Main Contribution

The original paper, Learning Cache Replacement with CACHEUS by Rodriguez et al., proposes CACHEUS as a class of adaptive machine-learned cache replacement algorithms. The framework builds on LeCaR, an earlier approach that used regret minimization to choose between LRU and LFU. CACHEUS keeps the two-expert learning idea but improves both the expert design and the learning-rate behavior.

The strongest variant in the paper uses two lightweight experts: SR-LRU and CR-LFU. SR-LRU is a scan-resistant version of LRU. It protects useful resident data from being evicted by long scans while still preserving recency benefits. CR-LFU is a churn-resistant version of LFU. It modifies LFU tie-breaking by evicting the most recently used item among equally least-frequent items, which can help lock a stable subset of pages in cache under churn.

3.2 Learning Mechanism

CACHEUS can be understood as an online learner that keeps a probability weight for each of two experts. On a miss that requires eviction, each expert proposes a victim. CACHEUS chooses one expert according to the current weights and evicts that expert's candidate. It also records metadata about evicted pages in histories. If a page is requested later while it is still in the history of one expert, CACHEUS interprets that event as evidence that the expert made a poor eviction decision and penalizes its weight.

This kind of feedback is delayed. At eviction time, the algorithm does not know whether the evicted page would have been reused soon. The history mechanism allows the algorithm to discover mistakes later. The use of two complementary experts is important because overlapping experts can confuse the learning process. CACHEUS is therefore not simply an ensemble of many algorithms; it is a two-expert framework where the choice of experts is central to performance.

3.3 Original Evaluation Scope

The original study is broad. It evaluates production storage I/O traces from five datasets: FIU, MSR, CloudPhysics, CloudVPS, and CloudCache. The paper reports 329 single-day workloads, six cache configurations, and 17,766 simulation experiments. Its primary claim is based on consistency across many workload and cache-size combinations, not only on a single average result.

The paper reports that the SR-LRU plus CR-LFU variant of CACHEUS is the best or statistically indistinguishable from the best-performing algorithm in most workload-cache combinations. It also reports that, when CACHEUS is worse, no single competing algorithm is consistently the best. This supports the interpretation of CACHEUS as a strong default policy across diverse workloads.

Original paper element	Description
Datasets	FIU, MSR, CloudPhysics, CloudVPS, CloudCache
Workloads	329 single-day production storage workloads
Cache configurations	6 sizes relative to workload footprint
Simulations	17,766 total experiments
Main variant	CACHEUS with SR-LRU and CR-LFU experts
Main claim	Most consistent across workload-cache combinations

4. Reproduction Study Design

4.1 Scope of This Study

This project is a reduced reproduction, not a complete rerun of the original paper. The active workflow focuses only on CloudVPS traces. It does not reproduce the FIU, MSR, CloudPhysics, or CloudCache portions of the original evaluation. It also does not reproduce the full paired t-test analysis over all workload-cache groups. The narrower scope makes the experiment more manageable while still allowing a meaningful comparison between CACHEUS and major baselines.

The reproduction evaluates 18 CloudVPS traces across five cache-size fractions: 0.001, 0.005, 0.01, 0.05, and 0.1. Six algorithms are tested: LRU, LFU, ARC, LIRS, LeCaR, and CACHEUS. This gives 18 traces times 5 cache sizes times 6 algorithms, or 540 total experiment rows. Cache-size fractions are interpreted relative to the number of unique expanded page-level requests in each converted trace.

Reproduction element	Value
Dataset	CloudVPS only
Traces	18
Algorithms	LRU, LFU, ARC, LIRS, LeCaR, CACHEUS
Cache-size fractions	0.001, 0.005, 0.01, 0.05, 0.1
Total runs	540
Main metric	Mean hit rate (%) and CACHEUS delta in percentage points

4.2 Research Questions

The reproduction is organized around three practical research questions. RQ1 asks whether CACHEUS remains competitive on CloudVPS traces when compared with classical and adaptive baselines. RQ2 asks how CACHEUS behaves across cache sizes, especially compared with LIRS, the strongest competitor in this reproduction. RQ3 asks what performance cost is visible in the Python simulator implementation and whether the additional complexity of CACHEUS appears justified by hit-rate gains.

These questions are narrower than the original paper's statistical research question. The original paper asks whether CACHEUS is the most consistent algorithm across many datasets and cache sizes. This study asks whether that story survives in one dataset family. The answer is nuanced: CACHEUS remains strong and beats most baselines, but LIRS is slightly better on the overall CloudVPS mean.

5. Project Pipeline Overview

The project pipeline is an offline trace-replay workflow. It starts with local CloudVPS block I/O trace archives and ends with CSV summaries and PNG figures. The active entrypoint is `cloudvps_pipeline.py`. The lower-level simulator runner is `code/run.py`. The trace reader expands converted block events into page-level requests, and each cache algorithm implements a request method that returns whether the request was a hit or an insertion/miss.

The pipeline deliberately avoids network downloads, databases, notebooks, containers, or web interfaces. This is useful for reproducibility because the full workflow can be repeated from local trace archives and source code. The most important external system tool is `blkparse`, which decodes binary `blktrace` data into readable text events.

Stage	Tool / file	Purpose
Input collection	Local CloudVPS archives	Provide raw block I/O traces
Extraction	<code>cloudvps_pipeline.py</code>	Safely extract <code>vps*.tar.gz</code> archives
Trace decoding	<code>blkparse</code>	Decode <code>.blktrace</code> files into text events
Event filtering	Python parser	Keep Q queue events and skip completion events
Format conversion	<code>cloudvps_pipeline.py</code>	Write simulator-compatible <code>.blk</code> traces
Request expansion	VisaTrace reader	Align and expand requests to 8-sector pages
Simulation	<code>code/run.py</code>	Replay traces through algorithms
Analysis	Python + NumPy	Compute summaries and deltas
Visualization	Matplotlib	Generate PNG figures

The most important semantic choice is filtering only Q events. Decoded block traces can contain both queue and completion-style events. Counting both would risk double-counting the same I/O. Keeping queue events gives a single request stream for cache simulation. Another important choice is cache sizing. The cache fraction is not based on archive size or raw sector count; it is based on the number of unique page-level requests after alignment and expansion.

The simplified pipeline can be summarized as: CloudVPS `blktrace` archives, safe extraction, `blkparse` decoding, Q-event filtering, `.blk` conversion, VisaTrace page expansion, experiment matrix execution, algorithm request replay, hit/miss collection, normalized CSV generation, summary computation, and plotting.

6. Technical Documentation of the Implementation

6.1 Repository Components

The repository has two main layers. The first layer is the user-facing CloudVPS pipeline. It handles raw input discovery, archive extraction, `blkparse` execution, conversion, simulator configuration, output normalization, and plotting. The second layer is the simulator and algorithm library. It contains `code/run.py`, `code/algs/get_algorithm.py`, individual cache algorithm implementations, and shared utility classes.

The active workflow is intentionally narrow. The original simulator may have supported more general trace configurations, but this reduced checkout is designed around CloudVPS `.blk` conversion. The default algorithm set is fixed to `lru`, `lfu`, `arc`, `lirs`, `lecar`, and `cacheus`. The default cache-size fractions are 0.001, 0.005, 0.01, 0.05, and 0.1.

Component	Role
cloudvps_pipeline.py	End-to-end CloudVPS orchestration
code/run.py	Generic simulator runner
code/algs/get_algorithm.py	Maps algorithm names to classes
code/algs/*.py	LRU, LFU, ARC, LIRS, LeCaR, CACHEUS, and other policies
code/algs/lib/traces.py	Trace reading, alignment, request expansion
code/algs/lib/dequedict.py	Ordered dictionary for recency lists
code/algs/lib/heapdict.py	Heap dictionary for LFU-style priority
results/cloudvps	Generated CSV summaries, logs, and figures

6.2 Trace Conversion Semantics

After blkparse decoding, a valid queue event contains a timestamp, action, operation, logical block address, plus sign separator, and request size. The converter rejects malformed lines, non-Q actions, invalid sizes, invalid LBAs, and nonfinite timestamps. A kept decoded line is written in the simulator format: timestamp 0 0 0 lba size op. The zero fields are placeholders so the existing VisaTrace reader can parse the line.

The VisaTrace reader uses an 8-sector page model. It aligns the LBA downward to the nearest page boundary and expands each request extent into one or more page-level requests. Therefore, one decoded block event can become multiple cache requests. All hit-rate measurements are therefore page-level measurements, not raw decoded-line measurements.

6.3 Algorithm Interface

Every algorithm class exposes a request method. For each page-level request, the simulator calls `alg.request(oblock, ts)`. The algorithm returns an operation code such as HIT or INSERT and an optional evicted object. INSERT represents a miss that inserts data into the cache. The simulator uses these operation codes to update hit, miss, write, and filter counters.

LRU uses a recency list and evicts the least recently used page. LFU uses a heap ordered by frequency, with implementation-specific tie-breaking. ARC maintains recency and frequency lists and adapts between them. LIRS uses reuse distance concepts and a HIR/LIR structure. LeCaR learns between LRU and LFU using regret minimization. CACHEUS in this repository implements the SR-LRU plus CR-LFU variant directly.

6.4 CACHEUS Implementation Details

The implemented CACHEUS policy maintains two resident partitions, often described as a protected or reuse-oriented area and a probationary scan-resistant area. It also maintains an LFU heap, eviction histories, expert weights, and adaptive learning-rate state. On a hit in the protected area, the page is refreshed and its frequency is increased. On a hit in the probationary area, the page is promoted and partition sizes may be adjusted. On a miss to a page found in history, the corresponding expert is penalized.

When eviction is needed, the SR-LRU side proposes the LRU item from the probationary partition. The CR-LFU side proposes the least-frequent item, with most-recent tie-breaking among equal-frequency pages. CACHEUS selects one candidate according to the learned weights. The evicted page metadata is stored in the selected expert history so future misses can reveal whether the decision was wrong.

This implementation is more expensive than simple LRU because it maintains several structures at once: recency partitions, an LFU heap, histories, expert weights, and learning-rate state. The runtime plots in this study should be interpreted as Python simulator costs rather than production-system overhead. Still, they are useful because they expose the engineering trade-off between adaptiveness and implementation complexity.

7. Experimental Results

7.1 Overall Mean Hit Rate

The first result is the overall mean hit rate across all CloudVPS traces and cache sizes. LIRS achieves the highest average hit rate at 28.08%. CACHEUS is second at 28.06%. The difference between these two algorithms is only about 0.017 percentage points, which is too small to treat as a meaningful practical gap in this reduced study. ARC, LeCaR, LRU, and LFU are lower.

This outcome is important because it prevents an oversimplified conclusion. CACHEUS does not dominate every metric in this reproduction. However, it remains very close to the strongest baseline and clearly improves over most other algorithms. The result is better described as a near tie with LIRS than as a failure of CACHEUS.

Algorithm	Mean hit rate (%)	Median hit rate (%)	Min (%)	Max (%)	Runs
LIRS	28.08	22.45	1.65	84.84	90
CACHEUS	28.06	23.37	2.74	84.97	90
ARC	26.54	22.02	2.64	82.85	90
LeCaR	25.44	19.79	2.46	84.20	90
LRU	22.13	15.56	2.47	81.61	90
LFU	20.03	17.07	1.10	58.96	90

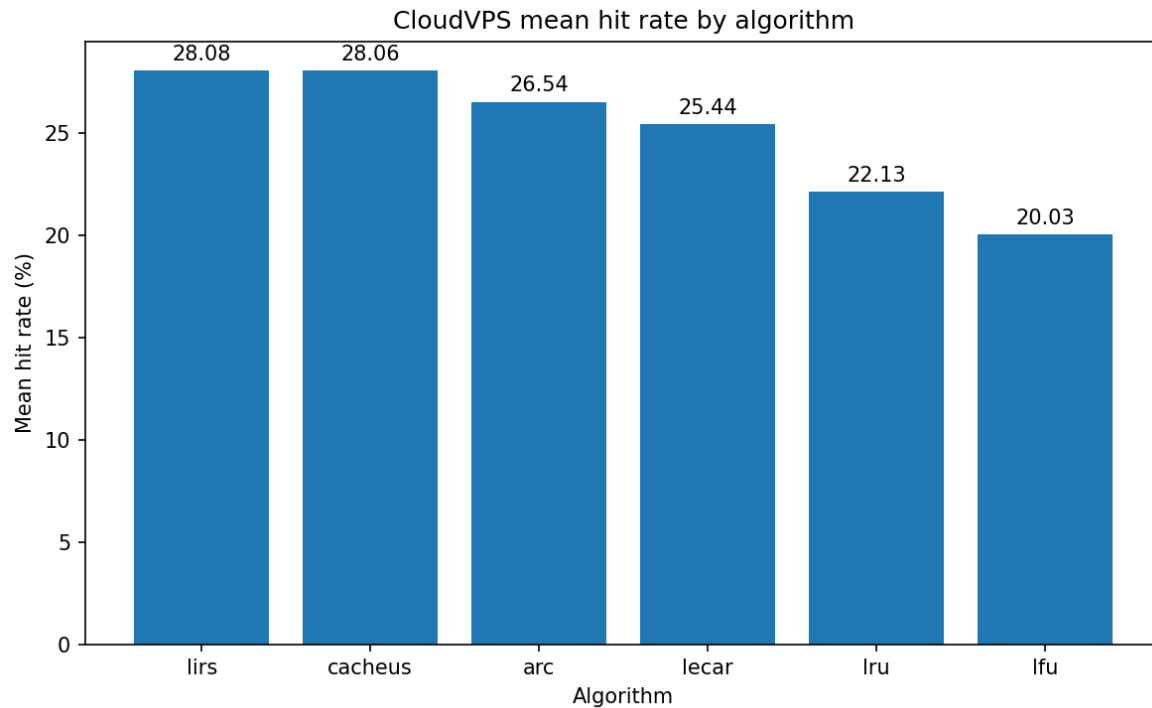


Figure 1. CloudVPS mean hit rate by algorithm. LIRS and CACHEUS are almost tied, while ARC, LeCaR, LRU, and LFU trail behind.

7.2 Hit Rate Across Cache Sizes

The cache-size line plot shows that hit rates generally increase as cache size increases. This is expected: a larger cache can retain more of the active working set. The important observation is that CACHEUS is among the top algorithms at every cache-size fraction, while LIRS becomes especially strong at the largest cache size.

At smaller cache sizes, the differences between algorithms reflect how well each policy handles limited capacity. LFU performs poorly at the smallest sizes because raw frequency is not enough to handle the complex mixture of access patterns. LRU also trails because recency alone cannot protect pages from churn and scans. ARC and LeCaR are stronger than the basic policies but remain behind CACHEUS and LIRS in the aggregate.

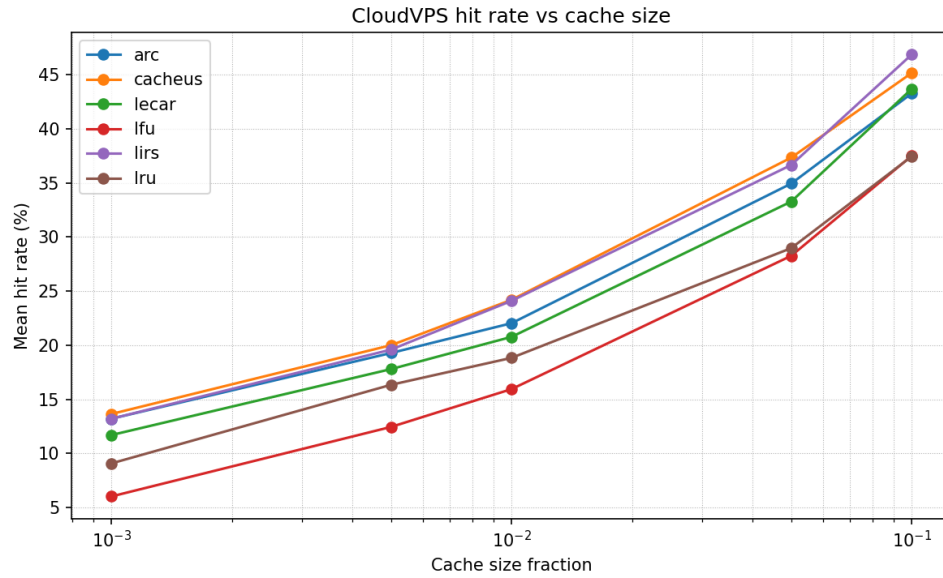


Figure 2. CloudVPS hit rate versus cache size. Hit rate increases with cache size, and CACHEUS remains near the top across the full range.

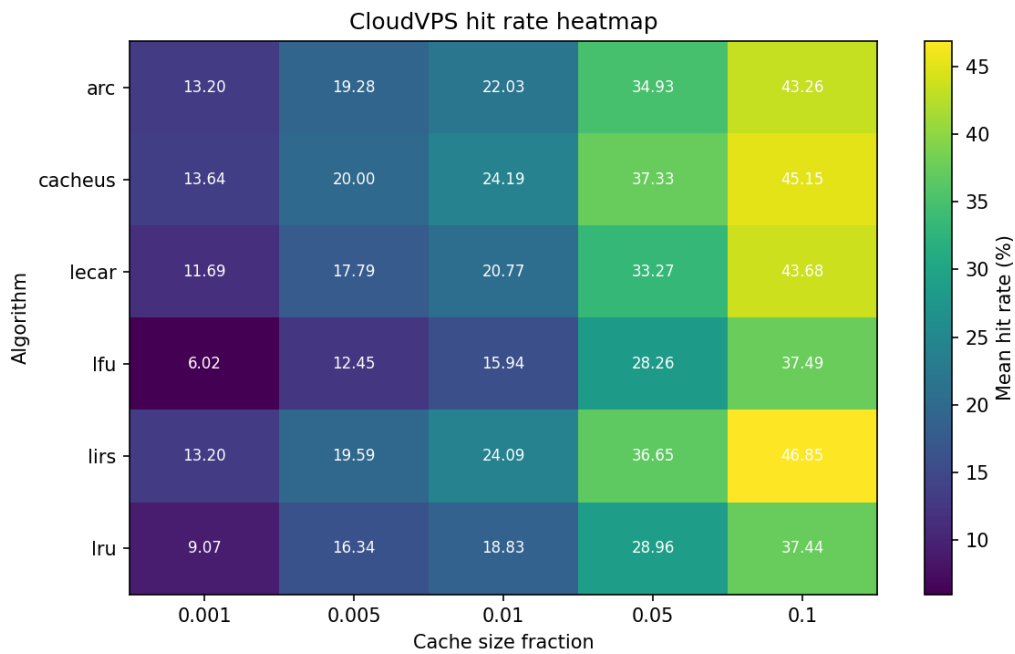


Figure 3. CloudVPS hit-rate heatmap. The heatmap highlights how algorithm ranking changes by cache-size fraction.

7.3 CACHEUS Delta Against Baselines

The delta analysis subtracts each baseline hit rate from the CACHEUS hit rate. A positive value means CACHEUS is better; a negative value means the baseline is better. The overall delta summary shows that CACHEUS improves over ARC by 1.52 percentage points, over LeCaR by 2.62 points, over LFU by 8.03 points, and over LRU by 5.93 points. Against LIRS, the overall delta is -0.017 points, meaning LIRS is ahead by a negligible amount.

The delta heatmap adds detail. CACHEUS beats LIRS at cache-size fractions 0.001, 0.005, 0.01, and 0.05, but trails LIRS at 0.1 by about 1.70 percentage points. This explains why the overall mean is so close. CACHEUS is often better than LIRS at smaller and middle sizes, but LIRS gains enough at the largest size to slightly overtake the overall average.

Baseline	Overall CACHEUS delta (percentage points)	Interpretation
ARC	+1.52	CACHEUS clearly better on average
LeCaR	+2.62	CACHEUS improves over earlier ML-based baseline
LFU	+8.03	CACHEUS much better than simple frequency
LRU	+5.93	CACHEUS much better than simple recency
LIRS	-0.017	Practical tie; LIRS slightly higher overall

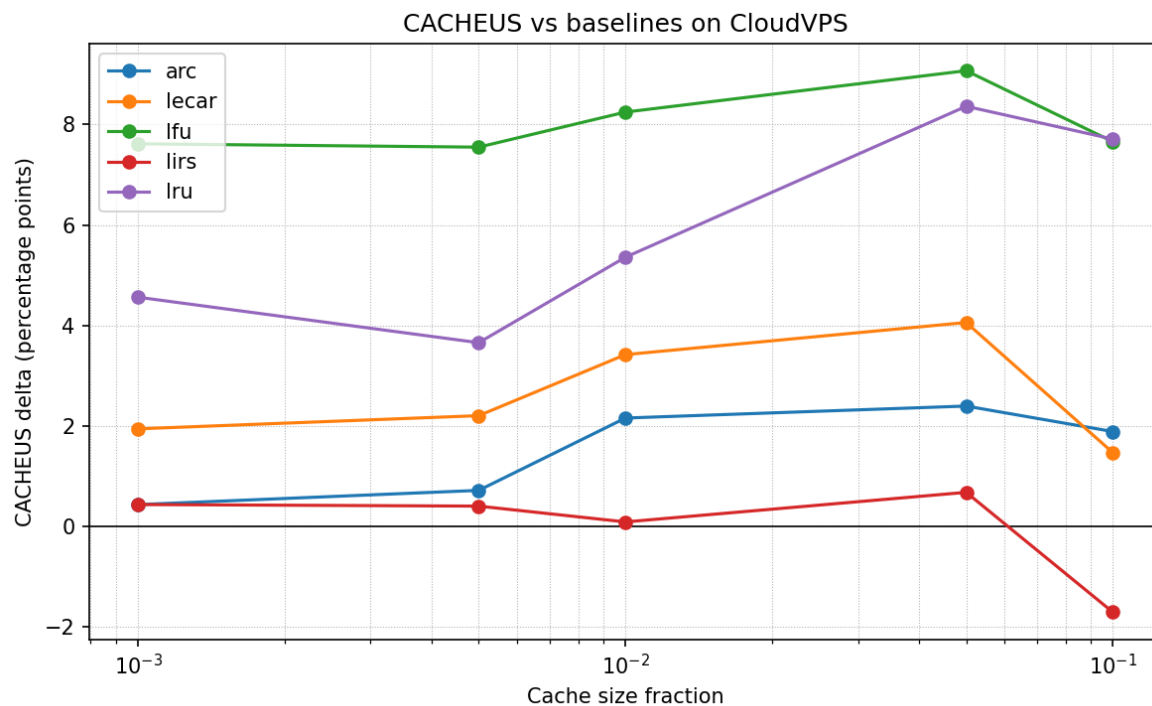


Figure 4. CACHEUS delta versus baselines across cache sizes. Positive values indicate that CACHEUS is better than the baseline.

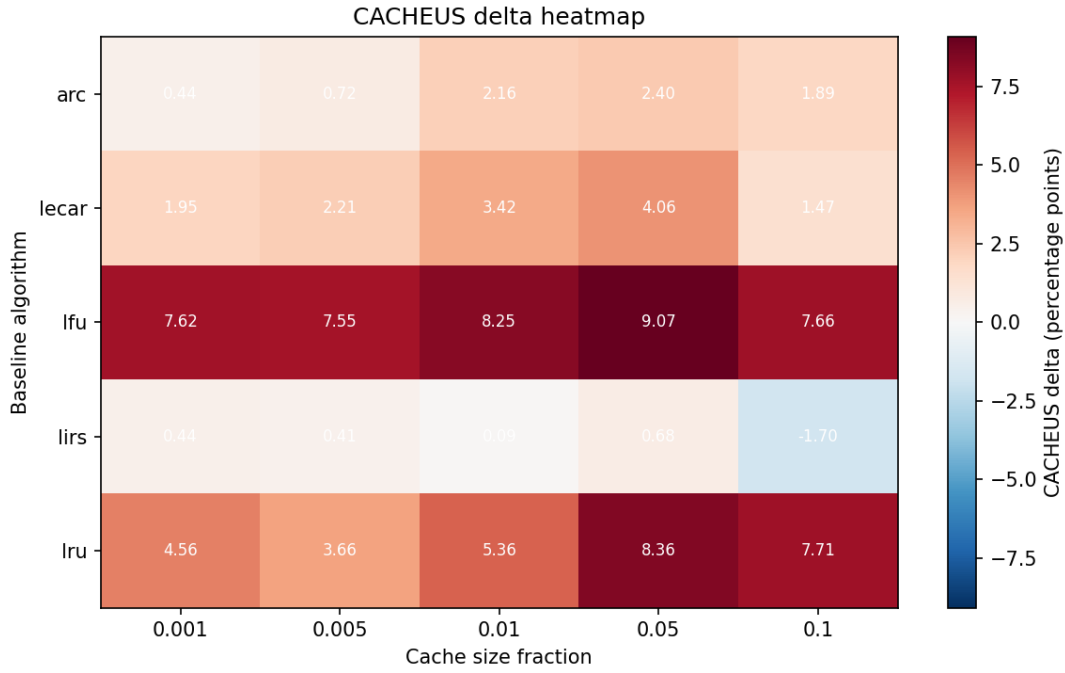


Figure 5. CACHEUS delta heatmap. CACHEUS beats LIRS at four cache sizes but loses at the largest cache size.

7.4 Delta Distribution

The boxplot of CACHEUS delta distributions shows variability across traces. The strongest positive distributions are against LFU and LRU, which confirms that CACHEUS handles CloudVPS access patterns better than simple recency or frequency alone. Against ARC and LeCaR, the median improvements are smaller but still generally positive. Against LIRS, the distribution is centered very close to zero, with both positive and negative cases.

This distribution-level view is useful because averages can hide trace-level behavior. A cache algorithm may perform very well on some traces and poorly on others. The boxplot suggests that CACHEUS is not simply winning because of one outlier. It consistently improves over the weaker baselines but faces a genuinely strong competitor in LIRS.

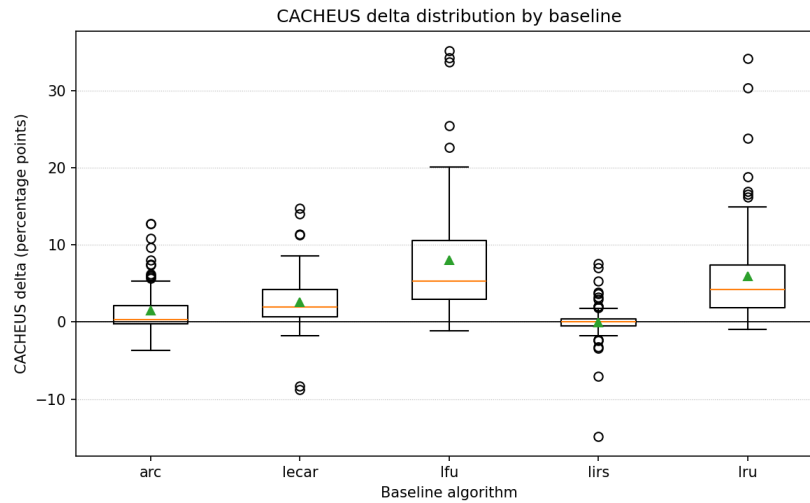


Figure 6. CACHEUS delta distribution by baseline. The distribution against LIRS is centered near zero, while deltas against LFU and LRU are strongly positive.

7.5 Direct CACHEUS vs. LIRS Comparison

Because LIRS is the closest competitor, the direct comparison between CACHEUS and LIRS is the most important figure in the reproduction. The plot shows that the two curves nearly overlap for most cache-size fractions. CACHEUS is slightly above LIRS at the smaller and middle sizes, while LIRS is above CACHEUS at the largest size. This single figure captures the main conclusion of the reproduction.

The result also fits the design of LIRS. LIRS is a strong scan-resistant algorithm based on reuse-distance concepts. If the CloudVPS traces contain patterns that LIRS handles well, then a near tie or slight LIRS advantage is not surprising. CACHEUS is designed for broad consistency, but a specialized or already strong policy can still win on a particular dataset family.

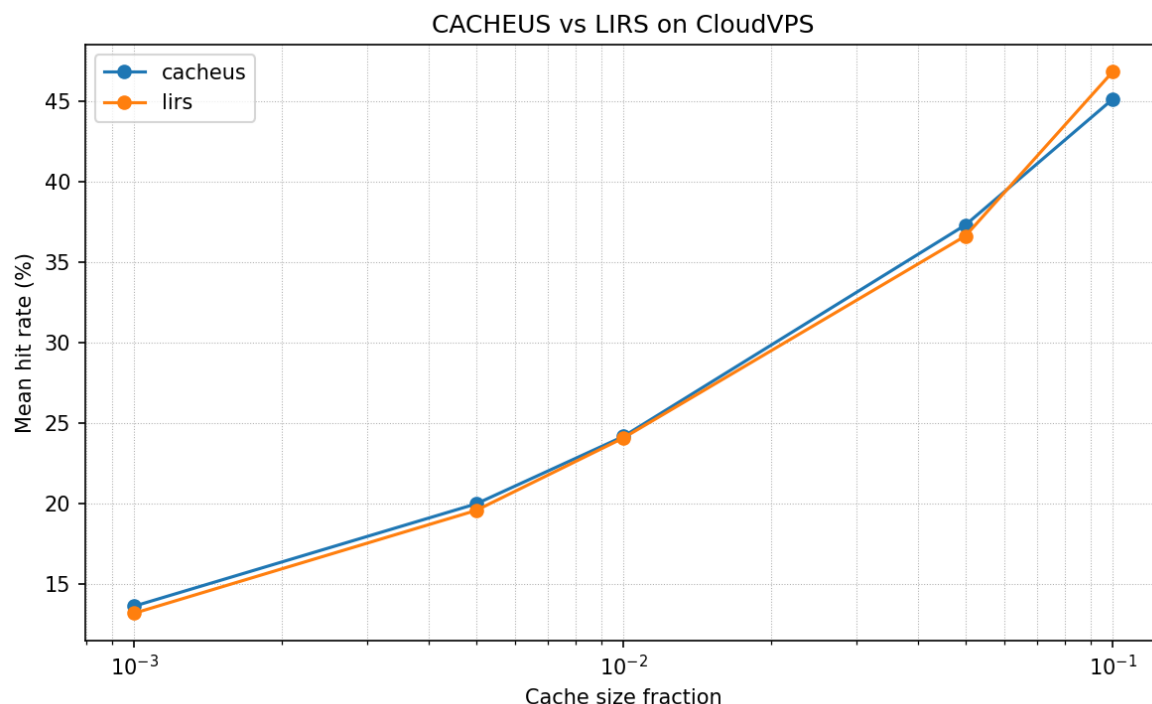


Figure 7. *CACHEUS* versus *LIRS* on *CloudVPS*. The curves are nearly identical until the largest cache size, where *LIRS* pulls ahead.

7.6 Runtime Results

Runtime is not the primary metric of the original CACHEUS paper, but it is relevant for an implementation study. In this Python reproduction, LRU is the fastest algorithm, followed by ARC and LIRS. CACHEUS is the slowest among the default algorithms. This is expected because CACHEUS maintains more metadata and performs more learning-related bookkeeping than simpler policies.

The runtime-versus-hit-rate plot reveals an engineering trade-off. LIRS reaches the best average hit rate with far lower runtime than CACHEUS in this reproduction. CACHEUS reaches nearly the same hit rate but with a higher simulator runtime. LeCaR is also relatively expensive but has a lower average hit rate. This does not prove that CACHEUS would be too slow in a production implementation; it only describes the cost in this Python simulator and codebase.

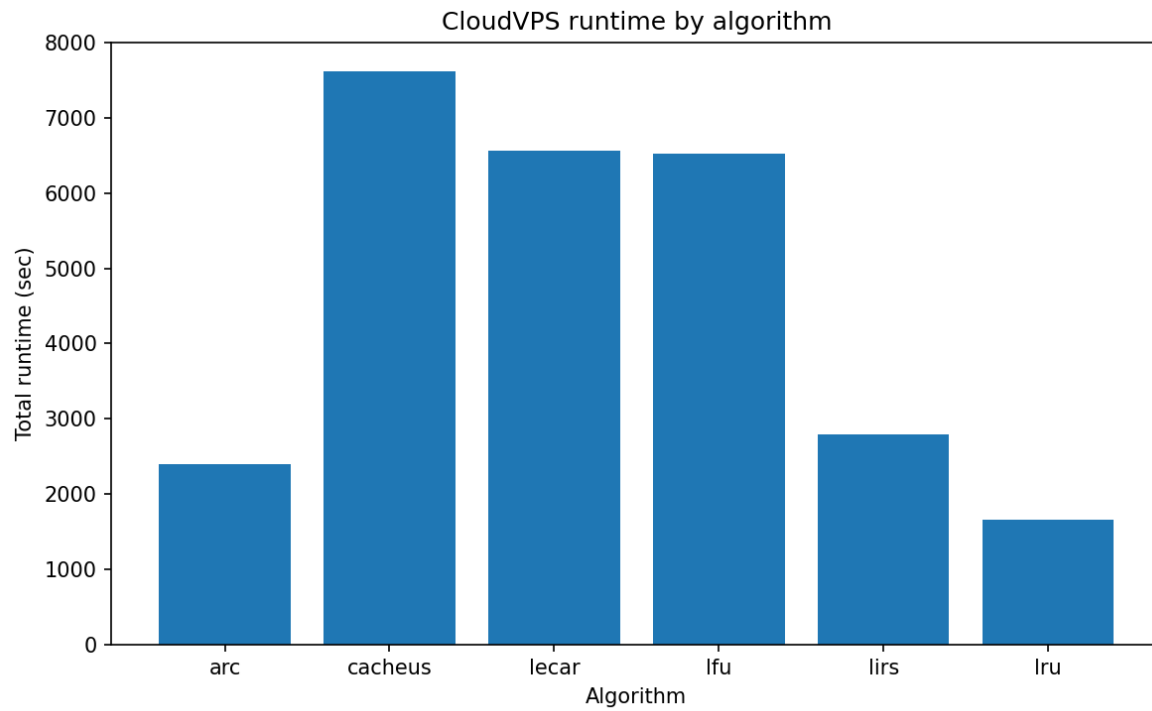


Figure 8. CloudVPS runtime by algorithm. CACHEUS is slowest in this Python implementation because it maintains multiple adaptive structures.

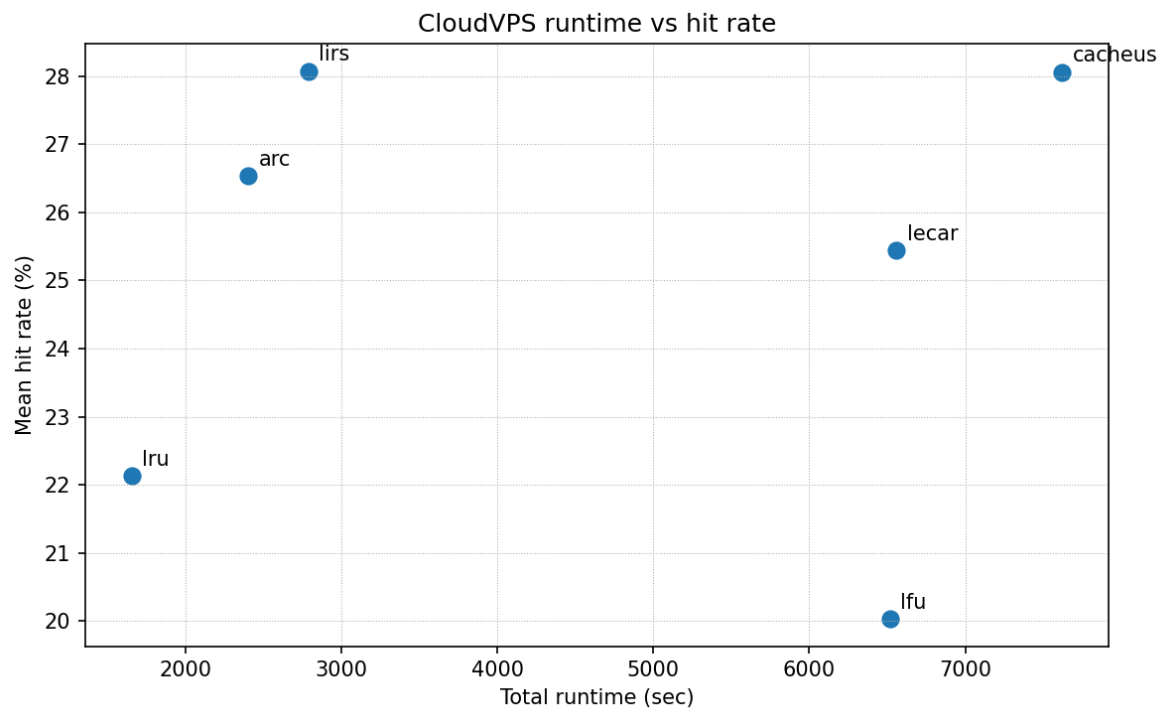


Figure 9. Runtime versus mean hit rate. LIRS gives the best runtime-hit-rate trade-off in this reproduction, while CACHEUS gives near-best hit rate with the highest runtime.

8. Interpretation and Discussion

8.1 What the Results Mean

The reproduction result should be interpreted carefully. It does not disprove the original CACHEUS paper. The original claim is about consistency across diverse datasets and cache sizes. This reproduction tests only CloudVPS. A policy can be the most consistent across many workload families while still being slightly behind another algorithm on one dataset family. In fact, the original paper itself argues that no single algorithm is best in every case.

The most accurate statement is that the CloudVPS reproduction partially supports the paper. CACHEUS is highly competitive, nearly tied with LIRS, and clearly above ARC, LeCaR, LRU, and LFU. However, the reproduction also shows that LIRS is an extremely strong baseline on these traces. Therefore, a presentation should avoid saying that CACHEUS wins outright. The honest conclusion is that CACHEUS behaves like a strong general policy, while LIRS is slightly better on the reduced CloudVPS average.

8.2 Why CACHEUS Beats Most Baselines

CACHEUS improves over LRU because it is not limited to recency. LRU can perform poorly when a scan pushes useful pages out of the cache or when churn repeatedly evicts pages that will soon be reused. CACHEUS improves over LFU because simple frequency does not handle recency shifts and can retain stale pages. CACHEUS improves over LeCaR because LeCaR uses only LRU and LFU as experts, while CACHEUS uses experts designed specifically for scan and churn resistance.

The improvements over ARC are smaller but still positive on average. ARC already adapts between recency and frequency using its internal lists and target parameter, so it is a stronger baseline than LRU or LFU. CACHEUS benefits from its explicit SR-LRU and CR-LFU experts and from its history-based blame mechanism. The average improvement over ARC suggests that these mechanisms help on CloudVPS, even if they do not beat LIRS overall.

8.3 Why LIRS Is So Close

LIRS is designed around reuse distance and is known to be strong against access patterns that expose the weakness of LRU. Its use of LIR and HIR categories gives it a way to protect pages with favorable reuse behavior while filtering less useful pages. On the CloudVPS traces, this seems to match the workload well enough that LIRS performs almost identically to CACHEUS and slightly better on the largest cache size.

The result also suggests that the CloudVPS dataset may not expose enough cases where CACHEUS's full adaptiveness dominates LIRS. In the original paper, the value of CACHEUS comes from diversity: different datasets, workload primitives, and cache sizes. A dataset subset with strong reuse-distance behavior can naturally make LIRS look excellent. That is why the reproduction should be framed as dataset-specific evidence rather than a universal ranking.

9. Original Paper vs. Reproduction

The original paper and this reproduction differ in scope, statistical depth, and goal. The original paper is a broad research evaluation. It introduces a new algorithmic framework, motivates it using workload primitive types, designs new experts, and uses statistical tests across a large experiment matrix. This reproduction is a narrower engineering and validation exercise. It asks whether the published idea remains visible when tested only on CloudVPS traces with a simplified pipeline.

The two results are compatible. The original paper says CACHEUS is the most consistent across a range of workloads and cache sizes. The reproduction says CACHEUS is nearly tied with LIRS on CloudVPS and clearly better than most other baselines. These statements can both be true. A strong general-purpose policy does not have to be the top policy on every individual dataset.

Dimension	Original CACHEUS paper	CloudVPS reproduction
Datasets	5 datasets	CloudVPS only
Workloads / traces	329 workloads	18 traces
Cache configurations	6	5
Algorithms	State of the art plus CACHEUS variants	LRU, LFU, ARC, LIRS, LeCaR, CACHEUS
Simulations	17,766	540
Statistical analysis	Paired t-tests and effect sizes	Mean hit rates, deltas, plots
Main conclusion	CACHEUS most consistent overall	CACHEUS near-tied with LIRS; better than most baselines

The reproduction also exposes a useful lesson about academic reproduction. Reproducing a paper does not always mean obtaining the exact same ranking in a reduced setting. It means rebuilding the experimental logic, checking whether the claims remain plausible under controlled changes, and being transparent about differences. Here, the reduced CloudVPS setting reveals both support for CACHEUS and an important dataset-specific exception: LIRS is slightly stronger on the overall mean.

10. Threats to Validity and Limitations

10.1 Dataset Scope

The largest limitation is dataset scope. Only CloudVPS is evaluated. The original paper uses five datasets. Therefore, this reproduction cannot evaluate the main cross-dataset consistency claim with the same strength. A complete reproduction would need to include FIU, MSR, CloudPhysics, and CloudCache traces with the same cache-size definitions used in the original paper.

The CloudVPS traces are still useful because they are part of the original evaluation and represent cloud-provider VM I/O behavior. However, they are only one workload family. The result should not be generalized to all storage systems without additional experiments.

10.2 Statistical Scope

This study reports means, medians, deltas, heatmaps, boxplots, and runtime plots. It does not reproduce the full paired t-test framework from the original paper. Therefore, claims about statistical significance are intentionally avoided. When this paper says that CACHEUS is practically tied with LIRS, that statement is based on the very small average difference and visual comparison, not on a formal hypothesis test.

A stronger future version should run paired tests across trace-cache groups and report effect sizes. This would make it possible to decide whether small differences such as 0.017 percentage points are statistically meaningful or simply noise from workload variation and algorithm randomness.

10.3 Implementation and Runtime Scope

The runtime measurements reflect a Python simulator implementation. They should not be interpreted as direct production-system costs. A carefully optimized implementation in C, Rust, or kernel-level code could change the runtime relationship between algorithms. The runtime results are still valuable for this project because they show the relative complexity of the implemented algorithms in the reproduction environment.

The codebase also contains quirks. For example, the reduced repository has no automated test suite, and some implementation details may differ from ideal textbook definitions. The generated results show that the pipeline runs successfully, but future research-quality work should add tests for trace parsing, request expansion, algorithm invariants, and deterministic output generation.

12. Conclusion

This paper documented a CloudVPS-only reproduction study of CACHEUS. The original CACHEUS paper proposes an adaptive, machine-learned cache replacement framework that learns between two complementary experts. Its strongest variant combines SR-LRU for scan resistance and CR-LFU for churn resistance. The original paper evaluates this design across hundreds of workloads and argues that CACHEUS is the most consistently strong policy across diverse workload-cache combinations.

The reproduction rebuilt a narrower pipeline around CloudVPS traces. Raw block traces were decoded with blkparse, filtered for queue events, converted to .blk format, expanded into page-level requests, and replayed through LRU, LFU, ARC, LIRS, LeCaR, and CACHEUS. The experiment produced 540 runs and generated CSV summaries and plots.

The final result is nuanced and useful. CACHEUS achieves an average hit rate of 28.06%, almost equal to LIRS at 28.08%. LIRS is ahead by only about 0.017 percentage points, which is practically a tie. CACHEUS still improves over ARC, LeCaR, LRU, and LFU by meaningful margins. Therefore, the reproduction partially supports the original paper: CACHEUS is a strong adaptive policy, but in this CloudVPS-only setting, LIRS remains an extremely strong competitor and slightly leads the overall mean.

For a presentation, the key message should be honest and precise: the original claim is about consistency, not universal dominance. This reproduction shows that CACHEUS remains highly competitive under a reduced CloudVPS workload, while also showing why careful baselines and dataset-specific interpretation matter.

References

- [1] Rodriguez, L. V., Yusuf, F., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., and Narasimhan, G. Learning Cache Replacement with CACHEUS. USENIX FAST 2021.
<https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [2] Rodriguez et al. Learning Cache Replacement with CACHEUS. PDF.
<https://www.usenix.org/system/files/fast21-rodriguez.pdf>
- [3] Vietri, G. et al. Driving Cache Replacement with ML-based LeCaR. USENIX HotStorage 2018.
<https://www.usenix.org/conference/hotstorage18/presentation/vietri>
- [4] Vietri, G. et al. Driving Cache Replacement with ML-based LeCaR. PDF.
<https://www.usenix.org/system/files/conference/hotstorage18/hotstorage18-paper-vietri.pdf>
- [5] Megiddo, N. and Modha, D. S. ARC: A Self-Tuning, Low Overhead Replacement Cache. USENIX FAST 2003.
<https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [6] Jiang, S. and Zhang, X. LIRS: An Efficient Low Inter-reference Recency Set Replacement Policy to Improve Buffer Cache Performance. ACM SIGMETRICS 2002. <https://dl.acm.org/doi/10.1145/511399.511340>
- [7] Arteaga, D. and Zhao, M. Client-side Flash Caching for Cloud Systems. ACM SYSTOR 2014.
<https://dl.acm.org/doi/10.1145/2611354.2611372>
- [8] ASU V.I.S.A. Lab block-level I/O traces page. <https://visa.lab.asu.edu/web/resources/traces/>
- [9] SyLab CACHEUS GitHub repository. <https://github.com/sylab/cacheus>
- [10] Shirin Manzari. Reproducibility Study of CACHEUS on CloudVPS. GitHub repository.
<https://github.com/shirin-manzari/reproducibility-study-of-CACHEUS>